NISR 2026 BIG DATA HACKATHON · TRACK 1: AGRICULTURAL PRODUCTIVITY CHALLENGE

CropHealth EWS

Micro-climate early warning for Late Blight and Early Blight in Musanze, Rwanda

Team

NIYOKWIZERA JEAN D AMOUR

UWASE YVONNE

Affiliation: African Institute for Mathematical Sciences (AIMS)

Live app: https://nisr-hackton-2026-crop-health.vercel.app/

Sign-in: admin / crophealth2026

Source code: GitHub repository (link in the root README)

Google Colab notebooks

Open in Colab Open in Colab

Demonstration video: `demo/demo.webm` (2 min 42 s, with captions in `demo/demo.vtt`).

Contents

1. Executive summary
2. Problem and national relevance (Criterion 1)
3. Data and methodology (Criterion 2)
4. Technology and innovation (Criterion 3)
5. Usability and design (Criterion 4)
6. Tangible impact (Criterion 5)
7. System architecture and deployment
8. Future work: IoT implementation roadmap
9. Limitations and honest scope
10. Reproducing the project

1. Executive summary

Smallholder farmers of potato, tomato, pepper, and eggplant in Rwanda lose yield to fungal diseases, mainly Late Blight (*Phytophthora infestans*, *Kiniga*) and Early Blight (*Alternaria solani*). These diseases develop when temperature, humidity, leaf wetness, and soil moisture reach particular levels. Most farmers spray on a routine schedule instead, which wastes money on chemicals, leaves disease uncontrolled when conditions are dangerous, and adds chemical runoff to watersheds.

CropHealth EWS turns micro-climate readings into a risk probability for each disease, a risk tier, and a recommended action. Farmers receive the advice as a Kinyarwanda SMS, so it works on basic phones without internet access. The system is a working prototype: a trained dual-output Random Forest model, a FastAPI backend, a Next.js dashboard, and Docker deployment on Render. It has not yet been tested in a field pilot.

2. Problem and national relevance (Criterion 1)

The problem. Farmers apply pesticides at the wrong times, or not at all when they are needed. In the NISR Seasonal Agricultural Survey (SAS) 2026 Season A, 74.11% of Musanze farmers reported applying chemical pesticides, and Dithane and Ridomil dominate treated plots. Without a local, timely signal of when their own field crosses the threshold for fungal growth, farmers spray by guesswork.

The response. The system forecasts outbreak risk from micro-climate conditions and tells the farmer what to do: a systemic fungicide in an emergency, a protective spray before the incubation window closes, or no spraying in safe conditions.

Alignment with national priorities

- **NST2 Priority Area 1 (Economic Transformation):** modernises production, reduces yield losses, and cuts input costs.

- **NST2 Priority Area 2 (Social Transformation):** protects smallholder livelihoods and reduces unnecessary agrochemical spending.
- **Vision 2050:** supports a climate-smart, productive agricultural sector.
- **PSTA 5 and MINAGRI plant health strategies:** the project plan includes alignment with these strategies.
- **RAB:** the project plan names RAB as technical partner for extension and disease surveillance.
- **Law No. 058/2021 on personal data protection:** the data plan keeps farmer identifiers separate from analysis data.

3. Data and methodology (Criterion 2)

3.1 Data sources

Source	Content	Use in the project
NISR SAS 2026 Season A tables	District pesticide adoption, chemical distribution, crop areas	Baseline context; the adoption rate (0.7411) is a model input
NASA POWER hourly agroclimate data	Temperature, humidity, precipitation, for Musanze (latitude -1.499, longitude 29.634), January to October 2026	Hourly micro-climate history
Unified dataset (`data/crophealth_unified_nisr_2026.csv`)	7,032 hourly records combining both sources with a foliar-wetness index	Training and dashboard telemetry

3.2 Model

- **Type:** Random Forest classifier with two outputs, one per disease (120 trees, maximum depth 8).
- **Inputs:** 14 engineered features, including 12-hour rolling statistics for humidity, temperature, soil moisture, and hours of leaf wetness.
- **Validation:** chronological 80/20 split. The test set is the final 1,407 hours, which the model never saw during training.
- **Preprocessing:** sensor flags are removed from rainfall data. Tree models need no feature scaling.

3.3 Reported results

These figures come from the team's training run on the held-out test set.

Target	Accuracy	ROC-AUC	Recall	F1
Late Blight	96.30%	0.9955	96.99%	0.9727
Early Blight	86.28%	0.9859	—	0.8765

For Late Blight, a missed outbreak can destroy a field, while a false alarm only triggers an inspection. The model therefore prioritises recall: it finds 96.99% of true outbreak hours.

3.4 What drives the prediction

Feature importances are computed from the saved model (mean decrease in impurity, averaged over both outputs).

Rank	Feature	Importance	Agronomic meaning
1	Wetness duration, past 12 h	32.1%	A continuous water film on the leaf lets zoospores infect
2	Leaf wetness (current)	11.1%	Current surface wetness
3	Soil moisture, 12-hour mean	10.5%	Waterlogged soil raises canopy humidity
4	Precipitation	8.7%	Rain refreshes the leaf film and spreads spores
5	Soil moisture (current)	7.3%	Current root-zone moisture
6	Hours of soil saturation, past 12 h	7.0%	Prolonged saturation
7	Hours at or above 85% humidity, past 12 h	5.8%	Keeps spores viable

> Wetness-related features account for most of the decision, which matches the biology: no free water on the leaf, no infection.

4. Technology and innovation (Criterion 3)

- **One model for two pathogens.** Late Blight favours cool, wet conditions (about 18 °C with humidity of 85% or more). Early Blight favours warm, humid conditions (about 24.5 °C). A single model covers both.
- **Decisions, not just scores.** Each prediction maps to a risk tier and an action (the thresholds are listed in models/README.md).
- **Kinyarwanda SMS advice.** Advisories are written in Kinyarwanda for feature-phone users. The SMS gateway is simulated in this prototype.
- **Sensor-ready design.** The model's inputs are the variables that field sensors measure, so hardware can be connected later without changing the model.
- **Modern, portable stack.** A stateless FastAPI service, a Next.js dashboard, token-based sign-in, and Docker images for both services.

5. Usability and design (Criterion 4)

The dashboard is built for an agronomist or extension officer using a laptop or phone. A sign-in page guards the screens. A sidebar separates the Risk Simulator from the Telemetry & NISR Data view and shows whether the API is reachable.

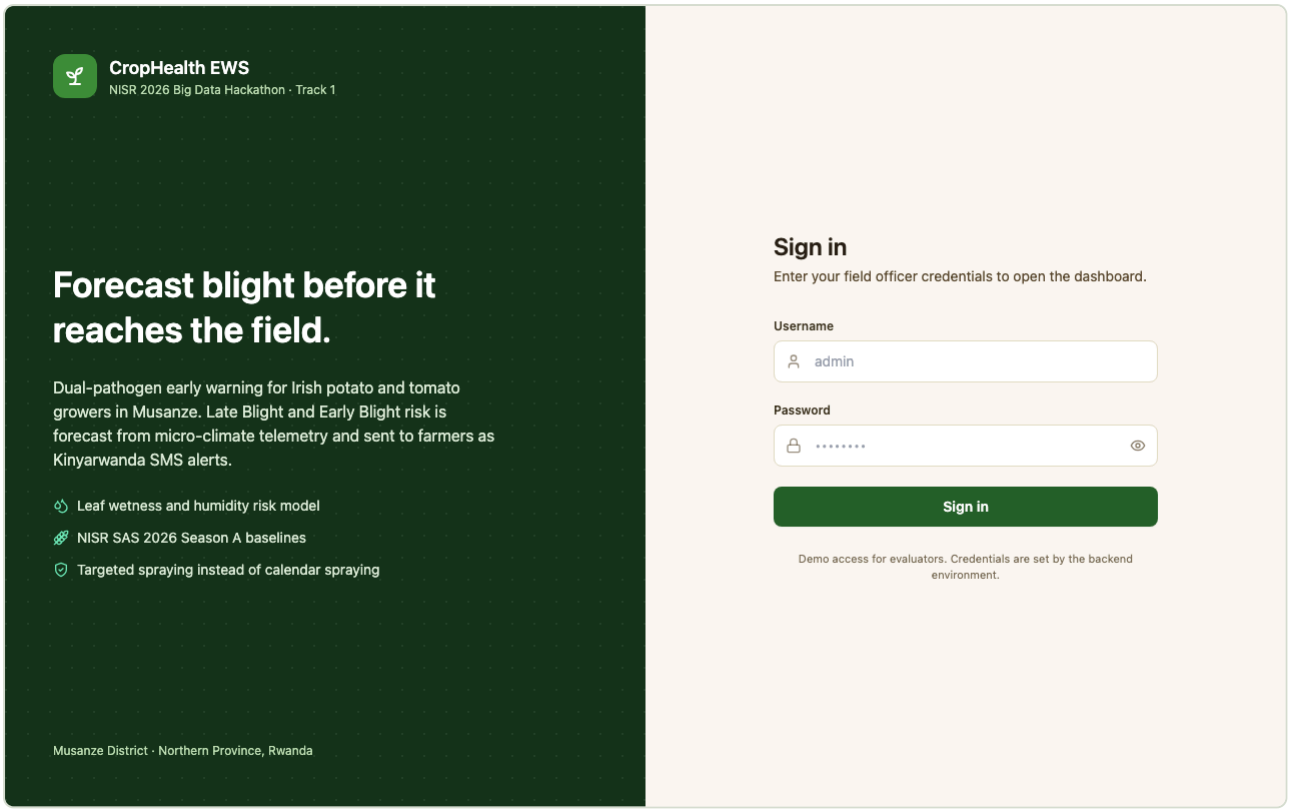


Figure 1. Sign-in page. Demo access for evaluators is listed in the README.

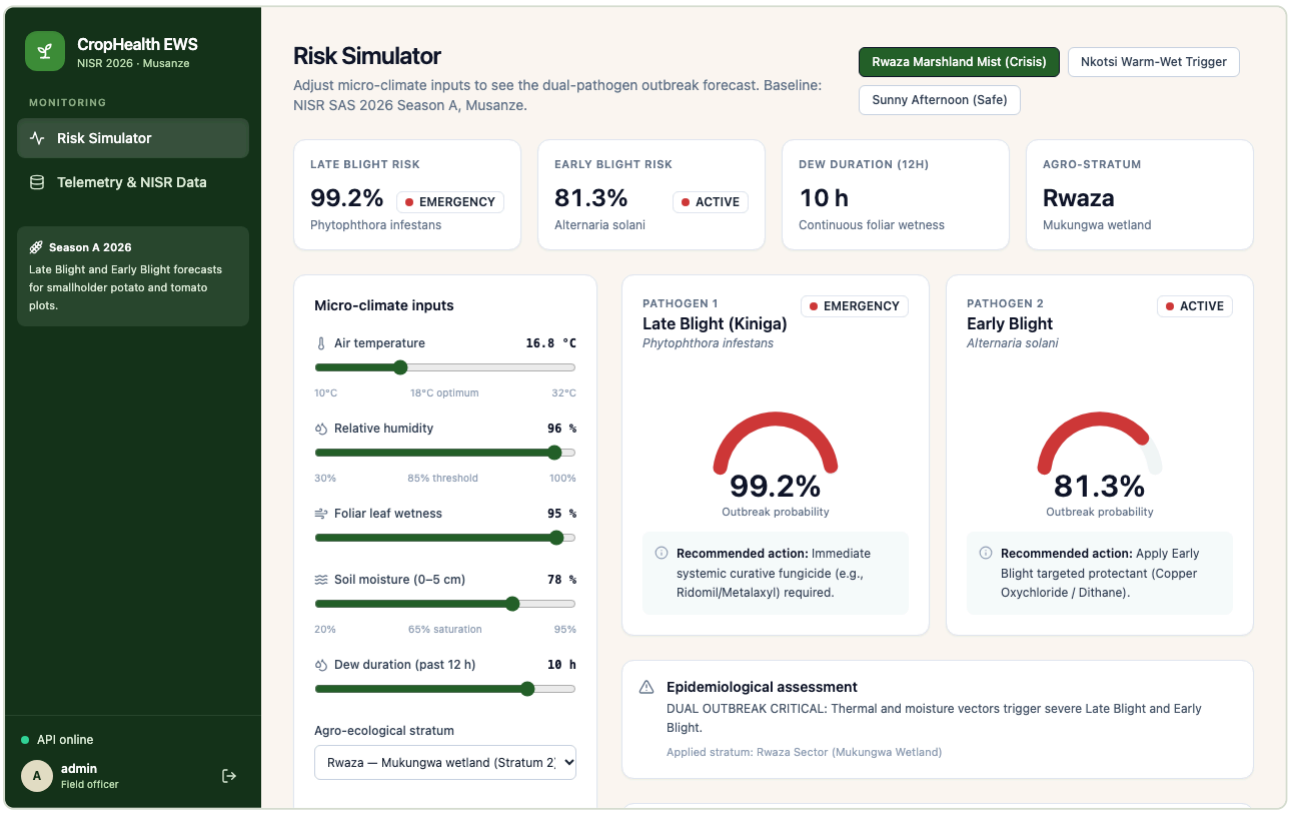


Figure 2. Risk Simulator after the Rwaza Marshland Mist (Crisis) preset. Late Blight is EMERGENCY (99.2%) and Early Blight is ACTIVE (81.3%), each with a recommended action.

- **Sliders and presets:** temperature, humidity, leaf wetness, soil moisture, dew duration, and stratum, with three one-click scenarios.

- **Gauges and tiers:** each disease shows a probability gauge, a labelled tier, and a recommended action. Status is never shown by colour alone.
- **SMS console:** shows the advisory as it would appear on a feature phone, with a delivery confirmation.

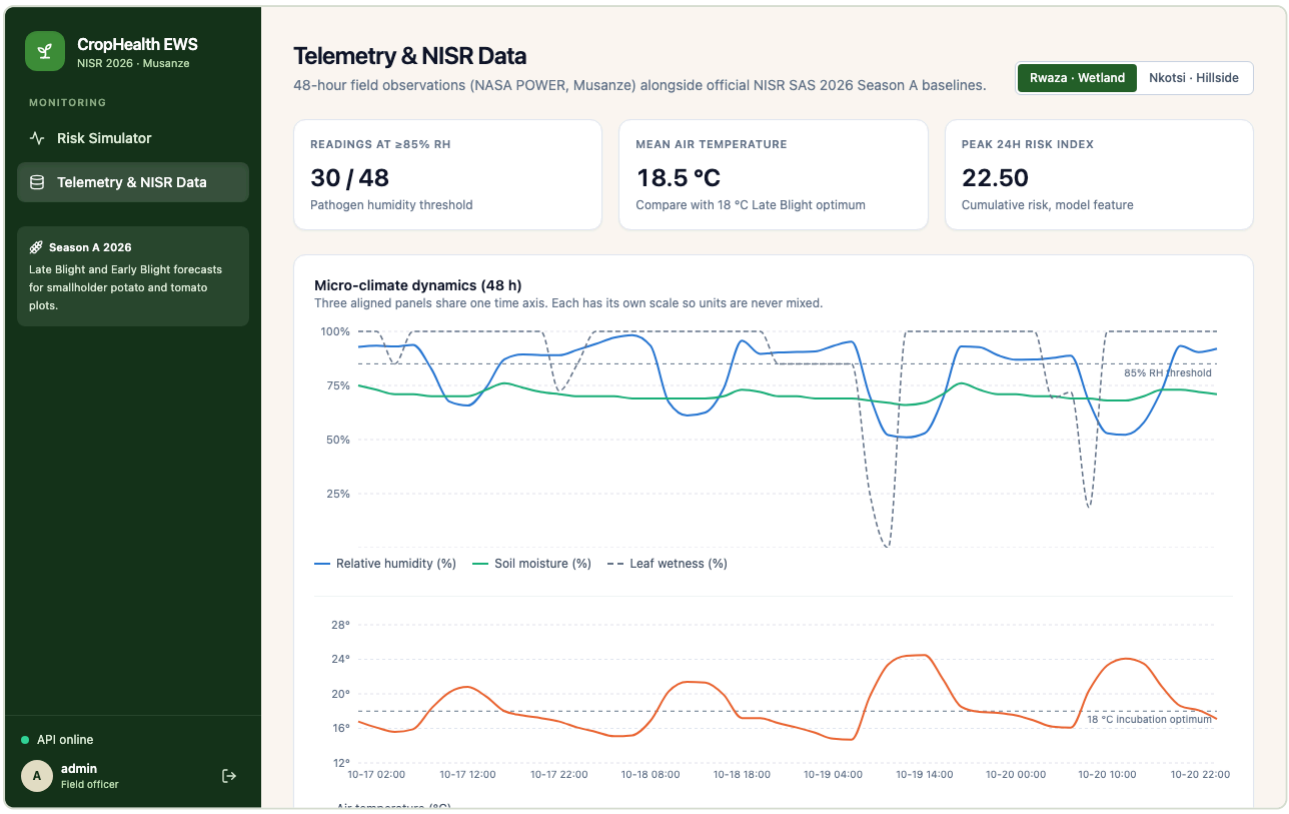


Figure 3. Telemetry & NISR Data view. Humidity, soil moisture, and leaf wetness share a time axis with the 85% humidity threshold; temperature has its own panel with the 18 °C incubation optimum.

> The feature-importance panel on the telemetry page currently shows fixed values. They are not yet computed from the model, and this is the next fix.

6. Tangible impact (Criterion 5)

Who can use it. Smallholder potato and tomato farmers, starting in the pilot districts of Musanze, Huye, and Kirehe (from the project proposal). Farmer cooperatives and district officers for field coordination. RAB and MINAGRI for surveillance and policy reporting. Telecom providers as the SMS channel.

Projected impact (estimates from the project proposal, not measured in this prototype):

- Skipping unnecessary calendar sprays during safe periods could reduce fungicide use by an estimated 35% to 50%, saving up to about 120,000 RWF per hectare per season.
- Targeted application reduces metalaxyl and chlorothalonil runoff into the Mukungwa River watershed.
- The SMS design reaches farmers who have no smartphone or internet.

- The approach can extend to Rubavu, Burera, and Nyabihu, and link to RAB's national monitoring data.

7. System architecture and deployment

Layer	Technology	Role
Dashboard	Next.js 14, TypeScript, Tailwind CSS, Recharts; static export served by nginx	Sign-in, simulator, telemetry, SMS console
API	FastAPI, Uvicorn, Python 3.12	Prediction, telemetry, NISR baselines, SMS advice, sign-in
Model and data	scikit-learn Random Forest (compressed, 0.9 MB); unified CSV	Inference and history
Hosting	Render (API as a Docker service); dashboard on Vercel	Public live link

The repository contains a Dockerfile for each service and a `docker-compose.yml` for local runs. The API reads its sign-in settings from environment variables, and the dashboard calls the API address set at build time.

8. Future work: IoT implementation roadmap

This section describes the planned field deployment. No sensors are installed in the current prototype.

8.1 Farm sensor kit (per farm)

- ESP32 microcontroller with GSM/Wi-Fi
- DHT22 temperature and humidity sensor (canopy height)
- DS18B20 soil temperature sensor
- Capacitive soil-moisture sensor (root zone)
- Grid-electrode leaf-wetness sensor
- 5 W solar panel and 3.7 V Li-ion battery; IP65 enclosure

8.2 Deployment and integration

1. Install sensors at canopy height and in the root zone, calibrate them against reference instruments, and record GPS coordinates.
2. Send readings over MQTT to the cloud every 15 minutes, with weekly quality checks for out-of-range values, gaps, and drift.
3. Store readings in a time-series database, and replace the simulator's single-reading approximation with real 12-hour history.
4. Run the existing prediction endpoint on live farm data. Its inputs already match the sensor variables.
5. Send SMS advisories to registered farmers, and accept disease reports by SMS or USSD.

6. Record outcomes (scouting results, farmer reports, spray decisions) and use them to retrain the model.

8.3 Field validation plan

- **Sites:** Musanze, Huye, and Kirehe, with 15 to 20 farms per district chosen with cooperatives.
- **Ground truth:** weekly scouting on a 0 to 9 severity scale, farmer reports, symptom photographs, and RAB lab confirmation of a subset of samples.
- **Baseline:** a structured survey of about 500 farmers on spraying practice, costs, yields, phone access, and language.

8.4 Model extensions

- Add Powdery Mildew (*Leveillula taurica*) and Anthracnose (*Colletotrichum* species).
- Move to the proposal's forecast: 24 to 72 hours of history predicting outbreak probability within 7 days.
- Add USSD for farmers without smartphones.

9. Limitations and honest scope

- **Prototype stage.** The system has not been tested with farmers in the field.
- **Simulator approximation.** The live simulator builds 12-hour features from one reading, so its outputs are indicative rather than identical to the offline test results.
- **Reproducibility.** The training script is not yet in the repository, so the reported metrics cannot be regenerated from the code alone.
- **Simulated SMS gateway.** Messages are logged, not sent through a real SIM800L modem.
- **Hard-coded feature panel.** The dashboard's feature-importance numbers are fixed values, not computed from the model.
- **Unverified projections.** The savings and impact figures come from the proposal and have not been measured.

10. Reproducing the project

1. Local with Docker: `docker compose up --build`, then open http://localhost:3000 and sign in with admin / crophealth2026.
2. Local without Docker: install `requirements.txt` and run `python app.py` for the API, then `npm install` and `npm run dev` in `frontend/`.
3. Hosted: deploy the Render Blueprint in `render.yaml`, then set the dashboard's API address (see the root README).

Model details, the full feature list, and the risk-tier thresholds are in `models/README.md`. Setup and deployment steps are in the root `README.md`.